INTERNSHIP PROJECT REPORT

AI-Powered Debt Relief & Financial Recovery Platform

An AI-Powered Web Platform for Automated Debt Analysis, Settlement Prediction & Negotiation

Domain	Google Cloud Generative AI
Team ID	SWTID-2026-2638
Team Size	1
Submitted By	Melpadi Chithra (Team Lead)
Repository	github.com/Chithra-1611/AI-Powered-Debt-Relief-Financial-Recovery-Platform

Submitted in partial fulfillment of the requirements for the Internship Program — 2026

1. INTRODUCTION

1.1 Project Overview

The AI Powered Debt Relief & Financial Recovery Platform is an intelligent, AI-driven web application designed to simplify and automate the debt management and financial recovery process for borrowers. The platform allows users to record and manage their loan details, analyze their overall financial health, generate AI-powered negotiation strategies, and receive adaptive, personalized settlement recommendations, all within a secure and modern digital environment.

The system is built using a modern JavaScript/TypeScript stack comprising React.js (Vite) for the client interface, Supabase (PostgreSQL, Authentication, Row Level Security, and Edge Functions) for backend services and data persistence, and the Google Gemini API for generative AI capabilities. Together, these technologies provide a responsive, scalable, and borrower-friendly platform with intelligent financial processing and real-time AI-assisted decision support.

Core Usage Scenarios

- Scenario 1 – AI-Powered Settlement Recommendation: A borrower adds loan details including outstanding amount, EMI, overdue duration, and monthly income. The system analyzes the financial profile and generates personalized settlement recommendations along with debt stress analysis and financial health insights.
- Scenario 2 – Intelligent Negotiation Letter Generation: A borrower selects a loan account and requests a negotiation strategy. The platform uses the Google Gemini AI model to generate professional, lender-specific negotiation emails and settlement request letters based on the borrower's financial condition and repayment capability.
- Scenario 3 – Financial Health Tracking & Loan Analysis: A borrower accesses the dashboard to monitor financial health metrics such as monthly surplus, EMI ratio, debt stress level, and settlement percentages, while the platform maintains a history of AI-generated negotiations and provides guidance for improving debt management decisions.

1.2 Purpose

The purpose of this project is to address the widespread and growing problem of unmanaged consumer debt by providing an accessible, private, and intelligent alternative to traditional debt settlement channels. Existing debt relief options are either prohibitively expensive (professional credit counsellors and legal advisors) or emotionally and financially damaging (ignoring creditor communication until legal escalation occurs). The AI Powered Debt Relief & Financial Recovery Platform bridges this gap by combining automated financial analysis with generative AI, enabling financially stressed individuals to understand their debt position clearly, receive realistic settlement strategies, and communicate with creditors confidently and professionally, without judgment or excessive cost.

- Provide borrowers with an objective, judgment-free understanding of their financial and debt stress position.
- Automate the generation of professional negotiation and settlement letters that would otherwise require expensive legal or financial consultation.
- Deliver actionable, AI-generated repayment and settlement strategies tailored to each borrower's income and liabilities.
- Build borrower confidence and trust through transparent, secure, and easy-to-understand financial tools.

2. IDEATION PHASE

2.1 Problem Statement

A well-articulated customer problem statement enables an understanding of the challenges borrowers face and empathy with their financial and emotional situation before designing a solution. The following problem statements were derived using the customer problem statement framework (I am / I'm trying to / but / because / which makes me feel):

I am	I'm trying to	But	Because	Which makes me feel
A middle-income working professional juggling multiple active loans, credit cards, and EMIs.	Create a structured, realistic debt repayment and settlement schedule to clear what I owe.	It takes a long time and is confusing to manually calculate settlement math and repayment horizons.	Spreadsheet tools are too rigid and there is no automated financial engine that customizes a strategy based on my income.	Overwhelmed and anxious.
A financially distressed individual facing sudden cash-flow issues and collection pressure.	Understand my financial stress level and draft professional, customized hardship or negotiation letters to lenders.	I don't know the proper financial or legal terminology required to communicate effectively.	Hiring a human financial advisor or debt lawyer is far too expensive for my current budget.	Intimidated and helpless.

Reference: Customer Problem Statement Template — miro.com/templates/customer-problem-statement

2.2 Empathy Map Canvas

An empathy map was created to understand the target users of the platform from four perspectives — what they Say, Think, Do, and Feel — in relation to the debt-related problems the project addresses.

Target Users

- The Stressed Borrower — individuals overwhelmed by credit cards or personal loans.
- The Silent Debtor — users seeking private, judgment-free financial planning.
- The Debt Consolidator — users trying to combine multiple high-interest debts.

Empathy Analysis — "The Overwhelmed Debtor" Persona

Aspect	Description
Think & Feel	Feels anxious and trapped by interest rates; wonders whether they will ever get out of debt.
Hear	Constant collection calls, well-meaning but unhelpful family advice, and mixed marketing from debt-settlement firms.
See	Piles of unpaid bills, rising interest charts, minimum-payment warnings, and confusing online forum advice.
Say & Do	Says they are overwhelmed and want a clear plan; ignores unknown calls, tracks debt manually, and researches anonymously.
Pain	Shame and embarrassment, fear of a ruined credit score, and anxiety-driven sleeplessness.
Gain	A clear path to financial freedom, a discreet automated plan, and growing confidence in

	repayment progress.

2.3 Brainstorming

A structured brainstorming and idea-prioritization session was carried out to explore possible solution directions before converging on the final concept.

Step 1: Problem Selection

The guiding problem statement: current debt-relief models fail consumers — who lack personalized, flexible pathways out of debt, leading to high default rates and financial anxiety — while manual, spreadsheet-based tracking offers no intelligent, adaptive guidance. The goal was to replace outdated, reactive settlement approaches with a proactive, AI-driven self-service platform.

Step 2: Idea Listing & Grouping

Candidate concepts were listed and grouped into common themes: Core Financial Analytics, AI-Generated Negotiation Content, Settlement Risk Scoring, Conversational AI Support, and Secure Multi-Tenant Data Storage — including a full web platform (FinRelief-style dashboard), a chatbot-only assistant, and a lender-facing risk portal.

Step 3: Idea Prioritization

Ideas were plotted on an Importance vs. Feasibility grid. "An integrated web platform combining a financial engine, AI settlement predictor, and AI negotiation letter generator" scored highest on both importance and feasibility and was selected as the implemented solution. Other high-value but longer-term ideas (bank-account auto-sync, credit-bureau integration, native mobile app) were retained as future-scope items.

Reference: Brainstorm & Idea Prioritization Template — mural.co/templates/brainstorm-and-idea-prioritization

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The customer journey map traces the borrower's experience across three key stages, capturing actions, touchpoints, thoughts, feelings, and opportunities for improvement.

Stage 1: Discovery & Onboarding

- Actions: Visits the landing page, registers an account, and signs in securely via Supabase Authentication.
- Touchpoints: Home page, Register/Login forms, protected-route redirect to the Dashboard.
- Thoughts & Feelings: "Can I trust this platform with my financial data?" — anxious but hopeful.
- Opportunity: Clear security messaging and a simple, low-friction sign-up flow.

Stage 2: Loan Entry & AI Analysis

- Actions: Adds loan and financial-profile details; requests a settlement recommendation and financial-health analysis from the Gemini-powered edge function.
- Touchpoints: Loans page, Settlement page, Health page, interactive dashboard charts (Recharts).
- Thoughts & Feelings: "Will this realistically reflect my situation?" — cautiously optimistic.
- Opportunity: Transparent scoring (financial health score, stress level, priority score) builds trust in the AI output.

Stage 3: Negotiation & Recovery Tracking

- Actions: Generates an AI negotiation letter/settlement email, downloads a PDF report, and reviews negotiation history over time.
- Touchpoints: Letter page, Reports page (jsPDF export), AI Chat Drawer, negotiation history log.
- Thoughts & Feelings: "I finally have something professional to send my lender." — relieved and empowered.
- Opportunity: Downloadable, ready-to-send documents and a persistent history reduce repeat effort and anxiety.

3.2 Solution Requirement

Functional Requirements

FR No.	Requirement (Epic)	Sub-Requirement
FR-1	User Registration & Auth	Email/password registration and login via Supabase Auth; protected routing for authenticated pages.
FR-2	Profile & Financial Profile Management	Maintain full name, occupation, phone, monthly income/expenses, and savings in a per-user financial profile.
FR-3	Loan Portfolio Management	Add, edit, delete, and list loans (loan name, bank, type, outstanding amount, EMI, interest rate, overdue months, status).
FR-4	AI Settlement Recommendation	Send loan and income data to the Gemini-powered edge function; receive settlement %, amount, stress level, financial-health score, negotiation points, and risk analysis.
FR-5	AI Negotiation Letter Generator	Generate a negotiation letter, settlement email, and formal hardship letter for a selected loan; persist to negotiation history.

FR-6	Financial Health Advisor	AI-generated budgeting tips, debt-reduction advice, and an overall financial summary based on income, expenses, savings, and EMI.
FR-7	AI Chat Assistant	In-app conversational assistant (AI Chat Drawer) for general debt-relief and financial-recovery questions.
FR-8	Dashboards & Reports	Visual dashboard (EMI distribution, income vs. expenses) and a downloadable PDF report of financial status via jsPDF.

Non-Functional Requirements

NFR No.	Requirement	Description
NFR-1	Usability	Clean, responsive Tailwind CSS interface with light/dark theming, accessible to financially stressed users.
NFR-2	Security	Supabase Row Level Security (RLS) on every table, scoping all reads/writes to auth.uid(); server-side Gemini API key never exposed to the client.
NFR-3	Reliability	Graceful error handling and toast notifications (react-toastify) for failed API/edge-function calls.
NFR-4	Performance	AI settlement and letter responses render within a few seconds under normal network conditions using the Gemini 2.0 Flash model.
NFR-5	Availability	Core dashboard and data features available at any time via Supabase's managed cloud infrastructure.
NFR-6	Scalability	Stateless React frontend and serverless Supabase Edge Functions allow horizontal scaling without architectural changes.

3.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a visual representation of the information flows within the system, showing how data enters and leaves the system, how it is transformed, and where it is stored.

DFD Level 0 — Overview

- External Entities: Borrower (User), Google Gemini AI Service.
- Processes: Supabase Auth (Registration/Login), Loan & Financial Profile Management, AI Settlement/Negotiation/Health Engine (Edge Function), Dashboard & Report Generation.
- Data Stores: profiles, loans, financial_profile, settlement_recommendations, negotiation_history (all in Supabase PostgreSQL with Row Level Security).
- Flow: The borrower authenticates via Supabase Auth → submits loan/financial data through the React client → the client calls the gemini-ai Supabase Edge Function → the edge function calls the Google Gemini API and returns structured JSON → results are persisted to PostgreSQL and rendered on the Dashboard, Settlement, Letter, and Health pages.

User Stories

ID	User Type	Epic	User Story / Task	Priority
US-1	Borrower	Onboarding & Auth	Implement Supabase email/password registration, session handling (AuthContext), and protected routing.	High

US-2	Active Borrower	Loan Portfolio	Build the Loans page with CRUD forms and a status badge (active / overdue / settled / closed).	High
US-3	Financial Analyst (self)	Dashboard Metrics	Compute and visualize EMI distribution, income vs. expenses, and financial health score with Recharts.	Medium
US-4	Distressed Borrower	AI Settlement Engine	Integrate the Gemini-powered settlement action to return percentage, amount, stress level, and priority score.	High
US-5	Distressed Borrower	Negotiation Letter Workspace	Integrate the letter action to generate a negotiation letter, settlement email, and lender hardship letter.	High
US-6	Borrower	Financial Health Advisor	Integrate the health action for budgeting tips and debt-reduction advice, shown on the Health page.	Medium
US-7	Borrower	AI Chat Assistant	Build the AIChatDrawer component wired to the chat action for free-form Q&A.	Medium
US-8	Borrower	Reports & History	Build the Reports page (PDF export via jsPDF) and a negotiation-history log backed by negotiation_history.	Low

3.4 Technology Stack

A well-chosen technology stack ensures the application is scalable, maintainable, and aligned with project requirements. The table below details the technologies actually used in the implemented codebase.

#	Component / Layer	Technology Chosen	Justification
1	Frontend / Client-Side	React 18 + TypeScript, Vite, React Router, Tailwind CSS	Fast dev/build tooling, type safety, and a component-driven, responsive UI with utility-first styling.
2	State & Auth Context	React Context API (AuthContext, ThemeContext), custom useData hook	Lightweight global state for the authenticated user, session, and light/dark theme without extra dependencies.
3	Backend / Database	Supabase (managed PostgreSQL, Auth, Row Level Security)	Zero-ops relational backend with built-in authentication and per-user data isolation enforced at the database level.
4	Serverless AI Layer	Supabase Edge Function (Deno) — gemini-ai	Keeps the Gemini API key server-side; a single function routes settlement, letter, health, and chat actions.
5	Generative AI	Google Gemini API (gemini-2.0-flash)	Produces structured JSON settlement analysis, negotiation documents, and conversational responses.
6	Visualization & Export	Recharts, lucide-react, jsPDF	Interactive charts for financial metrics, consistent iconography, and client-side PDF report generation.
7	Notifications & UX	react-toastify	Non-blocking success/error toasts for

			form submissions and AI responses.
8	Tooling & Quality	ESLint, TypeScript compiler (tsc --noEmit)	Static analysis and type-checking to catch errors before runtime.

4. PROJECT DESIGN

4.1 Problem – Solution Fit

The Problem-Solution Fit canvas was used to validate that the proposed solution genuinely addresses the identified customer problem and behavioral patterns.

Element	Details
Customer Segment(s)	Salaried professionals and gig workers overwhelmed by multiple EMI repayments; individuals with an EMI-to-income ratio exceeding 30–50% who are near default or facing active collection pressure.
Jobs-to-be-Done	Functional: understand debt severity and find an affordable path to clear it. Communication: generate a formal, authoritative negotiation letter to request settlement or restructuring.
Triggers	EMI bounce alerts, bank demand letters, collection calls, or overdue status on multiple loans.
Emotions (Before / After)	Before: isolation, shame, and silence about financial difficulty. After: clarity and relief through an objective, judgment-free view of debt stress.
Customer Constraints	Financial: limited funds for professional consultation. Psychological: anxiety, shame, and decision paralysis when facing lenders directly.
Available Alternatives	Expensive professional debt-settlement agencies, or avoidance of lender communication (temporary relief but severe long-term credit consequences).
The Solution	A self-service, privacy-first platform that lets a borrower assess debt objectively (financial engine + AI scoring) and generate ready-to-send negotiation content (Gemini-powered letters).

4.2 Proposed Solution

#	Parameter	Description
1	Problem Statement	Financially distressed borrowers struggle to assess the true severity of their debt and lack an affordable, judgment-free way to negotiate settlements or restructuring with lenders.
2	Idea / Solution	A React + Supabase + Google Gemini web platform that stores loan and financial-profile data securely, computes financial-health metrics, and generates AI-driven settlement recommendations and negotiation documents.
3	Novelty / Uniqueness	A single serverless edge function multiplexes four AI capabilities (settlement, letter, health, chat) against one Gemini model, combined with database-enforced Row Level Security so every user's financial data stays isolated by design.
4	Social Impact	Reduces shame and isolation for distressed borrowers by offering a private, non-judgmental, automated path to financial clarity and confident lender communication.
5	Business Model	Freemium access to financial-health tracking and dashboards, with a premium tier for unlimited AI negotiation-letter generation and advanced settlement analytics.
6	Scalability	Serverless Supabase Edge Functions and a managed PostgreSQL backend scale automatically with user load; the stateless React frontend can be deployed to any

		static hosting/CDN.

4.3 Solution Architecture

The solution follows a layered architecture comprising the Presentation Layer, Auth & Data Layer, and the Serverless AI Layer, as summarized below.

Architecture Layers

- Presentation / Client Layer: React 18 + TypeScript single-page application (Vite build) with React Router pages (Home, Login, Register, Dashboard, Loans, Settlement, Letter, Health, Reports, Profile) and Tailwind CSS styling.
- State & Session Layer: AuthContext manages the Supabase session and user; ThemeContext manages light/dark mode; useData centralizes data-fetching hooks.
- Data & Security Layer: Supabase PostgreSQL with Row Level Security — profiles, loans, financial_profile, settlement_recommendations, negotiation_history — each table scoped to auth.uid() with SELECT/INSERT/UPDATE/DELETE policies.
- Serverless AI Layer: A single Deno-based Supabase Edge Function (gemini-ai) routes settlement, letter, health, and chat actions to the Google Gemini API (gemini-2.0-flash) and returns parsed JSON.
- Export & Reporting: jsPDF generates downloadable financial reports client-side from the Reports page.

Component Description Table

Component	Description / Role	Technologies Used
Presentation Layer	Renders the Home, Auth, Dashboard, Loans, Settlement, Letter, Health, Reports, and Profile pages with a consistent DashboardLayout, PageHeader, SummaryCard, and StatusBadge components.	React, TypeScript, Tailwind CSS, lucide-react
Auth & Routing	Handles sign-up/sign-in, session persistence, and ProtectedRoute gating for authenticated-only pages.	Supabase Auth, React Router, AuthContext
Data Layer	Stores per-user loans, financial profile, settlement recommendations, and negotiation history with Row Level Security policies enforced at the database.	Supabase PostgreSQL, RLS policies, SQL migrations
AI Engine (Edge Function)	Single entry point (action: settlement | letter | health | chat) that builds prompts, calls the Gemini API, extracts JSON, and returns structured results with graceful fallback on parse failure.	Deno Edge Runtime, Google Gemini API (gemini-2.0-flash)
Visualization & Export	Dashboard charts for EMI distribution and income vs. expenses; client-side PDF export of financial reports.	Recharts, jsPDF

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

As a solo project, the work was executed using a personal Agile-inspired workflow. Work was organized into Epics, broken into user Stories, and estimated using Story Points on the Fibonacci scale (1 – Very Easy, 2 – Normal, 3 – Moderate, 5 – Difficult, 8 – Complex). The backlog below outlines the sprints executed to deliver the platform.

Sprint 1 — Project Setup & Authentication

Story	Task	Points	Priority	Owner
US-1	Vite + React + TypeScript project scaffolding, Tailwind configuration, routing skeleton.	3	High	Chithra
US-1	Supabase project setup, environment variables, Auth integration (Login/Register/ProtectedRoute).	5	High	Chithra

Sprint 2 — Database Schema & Security

Story	Task	Points	Priority	Owner
US-2	Design and migrate the profiles, loans, financial_profile, settlement_recommendations, and negotiation_history tables.	5	High	Chithra
US-2	Row Level Security policies (select/insert/update/delete) and updated_at triggers on every table.	5	High	Chithra

Sprint 3 — Loan Management & Dashboard

Story	Task	Points	Priority	Owner
US-2	Loans page: add/edit/delete loans with status badges and validation.	5	High	Chithra
US-3	Dashboard summary cards, EMI-distribution and income-vs-expenses charts (Recharts).	8	High	Chithra

Sprint 4 — AI Edge Function Integration

Story	Task	Points	Priority	Owner
US-4	Build the gemini-ai Supabase Edge Function; implement the settlement action with structured-JSON prompting and parsing.	8	High	Chithra
US-5	Implement the letter action (negotiation letter, settlement email, lender letter) and wire the Letter page.	8	High	Chithra
US-6	Implement the health action and the Health page's budgeting/debt-reduction tips.	5	Medium	Chithra
US-7	Implement the chat action and the AIChatDrawer component for conversational support.	5	Medium	Chithra

Sprint 5 — Reports, History, Theming & Polish

Story	Task	Points	Priority	Owner
US-8	Reports page with jsPDF export; negotiation-history log backed by negotiation_history table.	6	Medium	Chithra
US-8	Profile page, light/dark ThemeContext, toast notifications, loading states, and responsive polish.	5	Low	Chithra

Sprint 6 — Testing & Finalization

Story	Task	Points	Priority	Owner
US-8	Functional testing of all pages/actions, ESLint and TypeScript type-check cleanup.	5	Medium	Chithra
US-8	Build verification, environment-variable documentation, and deployment readiness.	3	Low	Chithra

Sprint Summary & Velocity

Sprint	Epic Focus	Total Story Points
Sprint 1	Project Setup & Authentication	8
Sprint 2	Database Schema & Security	10
Sprint 3	Loan Management & Dashboard	13
Sprint 4	AI Edge Function Integration	26
Sprint 5	Reports, History, Theming & Polish	11
Sprint 6	Testing & Finalization	8

Total story points delivered across all sprints: 76. With six sprints executed, the average velocity was approximately 12.7 story points per sprint (Total Story Points Completed ÷ Number of Sprints).

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Functional Testing

ID	Scenario	Expected Result	Result
FT-01	User Registration & Login Validation	Valid users register/log in successfully; invalid inputs show appropriate error messages.	Pass
FT-02	Loan & Financial Profile Input Validation	System accepts valid inputs and rejects invalid or incomplete values.	Pass
FT-03	AI Settlement Recommendation	Edge function returns a structured settlement percentage, amount, and stress level for a given loan.	Pass
FT-04	AI Negotiation Letter Generation	Edge function returns a negotiation letter, settlement email, and lender letter for a selected loan.	Pass
FT-05	Row Level Security Isolation	A user cannot read or modify another user's loans or negotiation history.	Pass
FT-06	PDF Report Export	Reports page generates and downloads a PDF summary without errors.	Pass

6.2 Performance Considerations

As a serverless, edge-function-driven architecture, performance was evaluated qualitatively against expected usage patterns rather than a dedicated load-testing tool. Key observations:

- The Gemini 2.0 Flash model was selected specifically for its low-latency responses relative to larger Gemini variants, keeping settlement and letter generation responsive for interactive use.
- Supabase Row Level Security policies are enforced at the database layer, adding negligible overhead while guaranteeing per-user data isolation without extra application-level filtering logic.
- The React client uses component-level loading states and toast notifications so that AI response latency (typically a few seconds) does not block the rest of the interface.
- Because the frontend is a static Vite build and the AI layer is a stateless Edge Function, both layers scale horizontally under Supabase's managed infrastructure without code changes.

7. RESULTS

7.1 Output Screenshots

The following core modules were implemented and demonstrated end-to-end: Home, Register/Login, Dashboard, Loans, Settlement Predictor, Negotiation Letter Generator, Financial Health Advisor, AI Chat Assistant, Reports, and Profile. Screenshots of each live page should be inserted here from the working application prior to final submission.

Demonstration of Proposed Features

Feature	Status	Demonstrated	Remarks
User Authentication & Protected Routing	Implemented	Yes	Supabase Auth with session persistence and route guarding.
Loan Portfolio Management	Implemented	Yes	Full CRUD with status tracking (active/overdue/settled/closed).
Dashboard & Financial Visualizations	Implemented	Yes	EMI distribution and income-vs-expenses charts render correctly.
AI Settlement Prediction	Implemented	Yes	Structured JSON output from Gemini persisted to settlement_recommendations.
AI Negotiation Letter Generator	Implemented	Yes	Three document variants generated and saved to negotiation_history.
Financial Health Advisor	Implemented	Yes	Budgeting tips and debt-reduction advice generated from live financial data.
AI Chat Assistant	Implemented	Yes	In-app drawer supports multi-turn conversation with the Gemini model.
PDF Report Export	Implemented	Yes	Client-side jsPDF export of the borrower's financial summary.

Overall, all 8 proposed core features were implemented and demonstrated, giving a 100% implementation rate against the original scope.

7.2 Code Quality & Reusability

The implemented codebase follows a modular architecture: reusable UI components (SummaryCard, StatusBadge, PageHeader, Modal, LoadingSpinner, ProtectedRoute), shared context providers (AuthContext, ThemeContext), a typed data-access layer (lib/types.ts, lib/supabase.ts), and a single, well-organized serverless AI entry point (supabase/functions/gemini-ai/index.ts).

Aspect	Rating (1–5)	Comments
Code Layout & Structure	4	Clear separation between pages, components, context, and lib utilities.
Readability	4	TypeScript interfaces (DatabaseLoan, SettlementResult, LetterResult, etc.) make data shapes explicit throughout.
Reusability	4	Shared components and a single multiplexed edge

		function minimize duplication across four AI features.
Documentation / Comments	3	Schema-level documentation is strong (SQL migration header); inline code comments could be expanded.
Overall Score	4	Project is functional, meets its objectives, and is suitable for academic/internship submission.

8. ADVANTAGES & DISADVANTAGES

Advantages

- Provides borrowers with an automated, objective, and judgment-free assessment of their financial and debt-stress position.
- Significantly reduces the cost and time barrier of obtaining professional negotiation and settlement letters through AI-generated drafting.
- A single serverless edge function cleanly multiplexes four distinct AI capabilities (settlement, letter, health, chat) against one Gemini model, keeping the AI layer easy to maintain.
- Database-enforced Row Level Security guarantees strict per-user data isolation without relying solely on application-level checks.
- Secure by design — Supabase-managed authentication, server-side API key handling, and typed data contracts throughout the React/TypeScript codebase.
- Modular, reusable frontend architecture (context providers, shared UI components, typed hooks) makes the platform easy to extend.
- All 8 planned core features were fully implemented and verified through testing.

Disadvantages

- Relies on predefined, manually entered financial inputs rather than real-time banking or credit-bureau data, which can limit the accuracy of settlement predictions.
- Gemini responses are parsed as JSON with a text-fallback path; malformed AI output in edge cases falls back to raw text rather than fully structured data.
- No direct integration with banks or financial institutions yet, requiring users to manually update loan and financial details.
- Automated SMTP email delivery of negotiation letters, native mobile apps, and machine-learning-based credit-score prediction remain pending/future features.
- As a solo-built project, code documentation and automated test coverage are currently limited, which may affect long-term maintainability without further refinement.

9. CONCLUSION

The AI Powered Debt Relief & Financial Recovery Platform successfully demonstrates how generative AI and automated financial analytics can be combined to address a real and pressing problem: helping financially distressed borrowers understand their debt situation and negotiate with creditors confidently, privately, and affordably. Across six self-managed sprints, the platform was designed, built, and tested as a full-stack solution comprising secure Supabase-backed authentication, financial-health analysis, an AI-powered settlement prediction engine, an AI-driven negotiation letter generator, and a conversational AI assistant, all built on React, TypeScript, Supabase, and the Google Gemini API.

Functional testing confirmed that all core modules — authentication, loan management, AI settlement prediction, negotiation letter generation, financial health advice, and PDF reporting — operate reliably and return the expected results. All 8 originally proposed core features were implemented and demonstrated successfully, reflecting a 100% delivery rate against the project's initial scope. While certain enhancements — such as real-time bank integration, automated email delivery, and a native mobile experience — remain as future work, the current solution stands as a stable, secure, and borrower-friendly platform that validates the core value proposition of AI-assisted debt relief.

10. FUTURE SCOPE

A phased roadmap has been identified to evolve the platform beyond its current state and address existing limitations.

Phase	Planned Enhancement	Timeline	Expected Impact
Phase 2	Integrate real-time banking APIs / statement parsing for automatic loan synchronization and financial analysis.	3–6 months	Improves data accuracy and reduces manual entry.
Phase 3	Automated SMTP email delivery of generated negotiation letters directly to lenders, with delivery tracking.	3–6 months	Removes a manual step and speeds up negotiation.
Phase 4	Multilingual negotiation support, a voice-enabled chat assistant, and predictive credit-score modeling.	6–12 months	More intelligent, personalized, and accessible experience.
Phase 5	Native mobile application and admin/analytics dashboard for aggregate (anonymized) debt-relief insights.	9–12 months	Broader reach and better product visibility.

11. APPENDIX

Source Code

Representative source code of the AI Powered Debt Relief & Financial Recovery Platform is provided below, organized by file. It includes the database schema (Supabase migration), the serverless AI layer (Supabase Edge Function), and key frontend files (routing, types, API client). Each block is followed by a short description of what that file implements.

Folder Structure

```
project/
├── src/
│   ├── App.tsx
│   ├── main.tsx
│   ├── index.css
│   ├── components/
│   │   ├── AIChatDrawer.tsx
│   │   ├── LoadingSpinner.tsx
│   │   ├── ProtectedRoute.tsx
│   │   ├── PageHeader.tsx
│   │   ├── Logo.tsx
│   │   ├── Modal.tsx
│   │   ├── SummaryCard.tsx
│   │   ├── StatusBadge.tsx
│   │   └── layout/DashboardLayout.tsx
│   ├── context/
│   │   ├── AuthContext.tsx
│   │   ├── ThemeContext.tsx
│   │   └── useData.ts
│   ├── lib/
│   │   ├── supabase.ts
│   │   ├── types.ts
│   │   ├── format.ts
│   │   ├── chartTheme.ts
│   │   └── export.ts
│   └── pages/
│       ├── Home.tsx
│       ├── Login.tsx
│       ├── Register.tsx
│       ├── Dashboard.tsx
│       ├── Loans.tsx
│       ├── Settlement.tsx
│       ├── Letter.tsx
│       ├── Health.tsx
│       ├── Reports.tsx
│       └── Profile.tsx
├── supabase/
│   ├── migrations/
│   │   └── 20260708130005_create_debt_relief_schema.sql.sql
│   └── functions/
│       └── gemini-ai/index.ts
├── package.json
├── vite.config.ts
├── tailwind.config.js
└── .env
```

Purpose: Overall project directory layout, separating the React/TypeScript frontend (src/) — pages, reusable components, context providers, and typed library utilities — from the Supabase backend (supabase/) containing the SQL schema migration and the Gemini-powered edge function.

package.json (dependencies)

```
"dependencies": {
  "@supabase/supabase-js": "^2.57.4",
  "jspdf": "^2.5.2",
```

```
  "lucide-react": "^0.344.0",
  "react": "^18.3.1",
  "react-dom": "^18.3.1",
  "react-router-dom": "^6.26.2",
  "react-toastify": "^10.0.5",
  "recharts": "^2.12.7"
}
```

Purpose: Pins the exact frontend dependencies — Supabase client, PDF export, icons, React, routing, toast notifications, and charting — ensuring a reproducible build.

supabase/migrations — Core Schema (excerpt)

```
CREATE TABLE IF NOT EXISTS loans (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL DEFAULT auth.uid() REFERENCES auth.users(id) ON DELETE CASCADE,
  loan_name text NOT NULL,
  bank_name text NOT NULL,
  loan_type text NOT NULL DEFAULT 'Personal',
  outstanding_amount numeric NOT NULL DEFAULT 0,
  emi numeric NOT NULL DEFAULT 0,
  interest_rate numeric NOT NULL DEFAULT 0,
  overdue_months integer NOT NULL DEFAULT 0,
  monthly_income numeric NOT NULL DEFAULT 0,
  status text NOT NULL DEFAULT 'active',
  due_date date,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

ALTER TABLE loans ENABLE ROW LEVEL SECURITY;

CREATE POLICY "select_own_loans" ON loans FOR SELECT
  TO authenticated USING (auth.uid() = user_id);
CREATE POLICY "insert_own_loans" ON loans FOR INSERT
  TO authenticated WITH CHECK (auth.uid() = user_id);
CREATE POLICY "update_own_loans" ON loans FOR UPDATE
  TO authenticated USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);
CREATE POLICY "delete_own_loans" ON loans FOR DELETE
  TO authenticated USING (auth.uid() = user_id);
```

Purpose: Defines the loans table and its Row Level Security policies. Every table in the schema (profiles, loans, financial_profile, settlement_recommendations, negotiation_history) follows this same owner-scoped pattern, so a user can only ever see or modify their own records.

supabase/functions/gemini-ai/index.ts (settlement action, excerpt)

```
const GEMINI_MODEL = "gemini-2.0-flash";
const GEMINI_URL = `https://generativelanguage.googleapis.com/v1beta/models/$
{GEMINI_MODEL}:generateContent?key=${GEMINI_API_KEY}`;

Deno.serve(async (req: Request) => {
  const body = await req.json();
  if (body.action === "settlement") return await handleSettlement(body);
  if (body.action === "letter") return await handleLetter(body);
  if (body.action === "health") return await handleHealth(body);
  if (body.action === "chat") return await handleChat(body);
  return json({ error: "Unknown action." }, 400);
});

async function handleSettlement(body) {
  const prompt = `You are a senior debt-settlement analyst. Return a JSON object ONLY:
  { "settlement_percentage": <0-100>, "settlement_amount": <number>,
    "debt_stress_level": "Low|Moderate|High|Critical",
    "financial_health_score": <0-100>, "repayment_strategy": <string>,
    "negotiation_points": [<string>], "priority_score": <0-100>,
    "risk_analysis": <string>, "advice": <string>, "recommendation": <string> }
  Borrower data: outstanding=$${body.outstanding_amount}, income=$${body.monthly_income}, ...`;
```

```
  const raw = await callGemini(prompt);
  return json(extractJson(raw));
}
```

Purpose: The single serverless entry point for all generative-AI features. It routes each request by action, builds a structured-JSON prompt for Gemini, calls the model, and safely extracts/parses the JSON response — with a graceful text fallback if parsing fails. The same pattern is repeated for the letter, health, and chat actions.

src/lib/types.ts (excerpt)

```
export interface DatabaseLoan {
  id: string;
  user_id: string;
  loan_name: string;
  bank_name: string;
  loan_type: string;
  outstanding_amount: number;
  emi: number;
  interest_rate: number;
  overdue_months: number;
  status: 'active' | 'overdue' | 'settled' | 'closed';
}

export interface SettlementResult {
  settlement_percentage: number;
  settlement_amount: number;
  debt_stress_level: string;
  financial_health_score: number;
  repayment_strategy: string;
  negotiation_points: string[];
  priority_score: number;
  risk_analysis: string;
  advice: string;
  recommendation: string;
}
```

Purpose: Centralized TypeScript interfaces mirroring the database schema and the edge function's JSON contracts (SettlementResult, LetterResult, DashboardStats, etc.), giving the entire frontend compile-time type safety when reading and rendering AI and database data.

src/App.tsx (routing, excerpt)

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/login" element={<PublicOnly><Login /></PublicOnly>} />
  <Route path="/register" element={<PublicOnly><Register /></PublicOnly>} />
  <Route path="/dashboard" element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />
  <Route path="/loans" element={<ProtectedRoute><Loans /></ProtectedRoute>} />
  <Route path="/settlement" element={<ProtectedRoute><Settlement /></ProtectedRoute>} />
  <Route path="/letter" element={<ProtectedRoute><Letter /></ProtectedRoute>} />
  <Route path="/health" element={<ProtectedRoute><Health /></ProtectedRoute>} />
  <Route path="/reports" element={<ProtectedRoute><Reports /></ProtectedRoute>} />
  <Route path="/profile" element={<ProtectedRoute><Profile /></ProtectedRoute>} />
  <Route path="*" element={<Navigate to="/" replace />} />
</Routes>
```

Purpose: The application's route table. Public routes (Home, Login, Register) redirect authenticated users to the Dashboard, while every data-bearing page is wrapped in ProtectedRoute to enforce an active Supabase session before rendering.

Dataset Link

This project does not use a pre-existing external dataset; all financial data (loan details, income, expenses) is entered directly by the user through the application interface and persisted in the Supabase PostgreSQL database, isolated per user via Row Level Security.

GitHub & Project Demo Link

- GitHub Repository: https://github.com/Chithra-1611/AI-Powered-Debt-Relief-Financial-Recovery-Platform
- Project Demo Video: https://drive.google.com/file/d/1xVMljKWnVqeZTEXmsQjVZbiJy2IoB-f1/view?usp=drivesdk

Team Details

Name	Role
Melpadi Chithra	Team Lead — Full-Stack Developer (Frontend, Supabase Backend, AI Integration)